

Communication Profiling and Characterization of Deep-Learning Workloads on Clusters With High-Performance Interconnects

Ammar Ahmad Awan, Arpan Jain,
Ching-Hsiang Chu, Hari Subramoni, and
Dhableswar K. Panda
The Ohio State University

Abstract—Heterogeneous high-performance computing systems with GPUs are equipped with high-performance interconnects like InfiniBand, Omni-Path, PCIe, and NVLink. However, little exists in the literature that captures the performance impact of these interconnects on distributed deep learning (DL). In this article, we choose Horovod, a distributed training middleware, to analyze and profile various DNN training workloads using TensorFlow and PyTorch in addition to standard MPI microbenchmarks. We use a wide variety of systems with CPUs like Intel Xeon and IBM POWER9, GPUs like Volta V100, and various interconnects to analyze the following metrics: 1) message-size with Horovod's tensor-fusion; 2) message-size without tensor-fusion; 3) number of MPI/NCCL calls; and 4) time taken by each MPI/NCCL call. We observed extreme performance variations for non-power-of-two message sizes on different platforms. To address this, we design a message-padding scheme for Horovod, illustrate significantly smoother allreduce latency profiles, and report cases where we observed improvement for end-to-end training.

Digital Object Identifier 10.1109/MM.2019.2949986

Date of publication 30 October 2019; date of current version

14 January 2020.

■ **HIGH-PERFORMANCE COMPUTING (HPC)** clusters are increasingly getting equipped with fast interconnects like Mellanox InfiniBand and Intel Omni-Path to offer low-latency and high-bandwidth communication across multiple compute nodes. These *internode* network interfaces are complementary to *on-node* interconnects like PCIe. On the other hand, we are witnessing an increase in the adoption of heterogeneous HPC systems that are equipped with fast GPUs like NVIDIA Volta V100 that have led to the rise of new and proprietary *on-node* interconnects like NVIDIA's NVLink. The primary goal of these interconnects is to increase the speed of data movement and keep GPUs' computation units busy, thereby avoiding stalls and getting the best performance. This is further propelled by emerging application areas like distributed deep learning (DL) that require intra- and internode communication for gradient aggregation during deep neural network (DNN) training. These applications require high bandwidth and low-latency communication to scale efficiently.¹

To this end, several studies have been published that deal with the performance of different interconnects using microbenchmarks and scientific applications.^{2–5} However, little exists in the literature that addresses the performance impact of modern interconnects on emerging application areas like distributed DL. High-level performance studies only focus on end-to-end performance⁶ but do not offer insights into low-level performance characteristics. In an early version of this article,⁷ we also identified a major problem for communication libraries like NCCL and MPI, i.e., non-power-of-two message sizes being used for gradient communication that lead to performance instabilities. Furthermore, existing studies are using a black-box approach, which only provides a limited and isolated view of performance. Thus, it is challenging to correlate microbenchmark performance (e.g., `osu_allreduce` from OSU Micro-Benchmarks (OMB) with end-to-end performance (e.g., training a TensorFlow or PyTorch model). To address these problems, we focus on the following broad challenge: How to effectively profile the performance of DL workloads on various

This is the first study that addresses performance analysis of a relatively new application area, i.e., distributed DL on HPC systems, with an emphasis on the impact of various high-performance interconnects.

high-performance interconnects and correlate low-level performance statistics to high-level application-specific metrics.

Along these lines, we investigate the following concrete challenges in this article.

- How do we correlate low-level performance statistics such as the latency for Allreduce with high-level metrics such as images per second for TensorFlow and PyTorch?
- Can we design an application-level profiling infrastructure that allows us to obtain such characteristics for mainstream communication middleware like MPI and NCCL for CPU and GPU based platforms?
- How do we improve performance of Allreduce for non-power-of-two message sizes that are being used due to techniques such as Horovod's tensor fusion?

Contributions

To the best of our knowledge, this is the first study that addresses performance analysis of a relatively new application area, i.e., distributed DL on HPC systems, with an emphasis on the impact of various high-performance interconnects. We make the following key contributions in this article.

- We propose and design application-level profiling infrastructure called *huprof* that can capture characteristics of DL workloads for various underlying interconnects and communication libraries.
- We analyze low-level performance characteristics of models like ResNet-50/101 and Inception-v3/4 on CPU- and GPU-based HPC systems.
- We identify inconsistent performance when nonpower-of-two message sizes are used and propose a padding scheme to improve the performance for such cases.
- We provide end-to-end application performance trends for TensorFlow and PyTorch and correlate them with low-level statistics to guide designers of MPI/NCCL libraries as well as DL frameworks.

CHALLENGES IN PROFILING DL WORKLOADS ON HPC SYSTEMS

We highlight the key challenges that we have addressed in this article.

Profiling DL Workloads on Various Interconnects

Current general-purpose profiling tools such as MPI Profiling (*mpiP*) and NVIDIA Profiler (*nvprof*) do not support profiling communication of new libraries like NCCL. We note the *nvprof* profiles NCCL calls but considers it as a computation kernel, which does not help understand the communication characteristics of a *ncclAllreduce* operation. *mpiP* offers profiling data on MPI operations like point-to-point and collective operations whereas *nvprof* profiles CUDA APIs, kernel activities, and some rudimentary information about MPI operations via its annotation feature, i.e., *-annotate-mpi*, by utilizing NVIDIA Tools Extensions (NVTX). Thus, there is a need for a more white-box approach to profiling different activities at the training middleware (e.g., Horovod) level. The biggest challenge in profiling GPU-based training with existing tools is an approach called *interception* using *LD_PRELOAD* for MPI calls. This is not supported by frameworks like TensorFlow that use low-level memory management infrastructure and thus cannot be profiled using *mpiP* and *nvprof -annotate-mpi*. We observed various failures, including errors like *CUDA_INVALID_CONTEXT* for such cases.

Profile Workloads in a DL-Framework Agnostic Fashion

Horovod, a distributed training middleware that supports distributed training for TensorFlow, PyTorch, MXNet, and Keras, is gaining considerable attention from ML as well as HPC communities. It is evident from the emerging distribution strategy APIs being developed by TensorFlow and PyTorch teams, both of which are very similar to Horovod's APIs. The main challenges are as follows. 1) How do we profile different kinds of communication calls used by Horovod? 2) Do we need to separate the statistics gathered due to a DL framework's call to allreduce operation from what is required by Horovod's multithreaded communication and synchronization engine? and 3) How do we differentiate between different MPI and NCCL calls?

Correlate End-to-End Application-Level Performance and Low-Level Profiling Data

The broad goal of our profiling and performance analysis study is to identify bottlenecks and improve performance for an emerging and important class of workloads. Our analysis needs to be helpful for two different communities: 1) DL software developers and 2) communication runtime designers. Both should be able to gain meaningful data so they can improve performance at their respective levels. For instance, images/second is a representative metric for DL developers to understand the performance of their DNN architecture but of little use for an MPI library designer. On the other hand, the latency of an MPI *Allreduce* operation can be used by an MPI library designer to optimize collective communication algorithms and design schemes but may not be very helpful for a DNN designer. The profiling analysis thus needs to cater to both communities with diverse requirements.

PROPOSED DESIGN

Profiling and Characterization

hvpof is our proposed infrastructure built for Horovod that enables the users to perform in-depth profiling of various workloads for several popular toolkits like TensorFlow, PyTorch, MXNet, and Keras. It provides multiple types of metrics that are valuable for both ML/DL developers as well as for designers of an MPI library. Because we implement *hvpof* at the Horovod level, we do not need interception/*LD_PRELOAD*. We maintain various profiling variables using Horovod Engine's global state. The following concrete statistics are gathered through function calls to MPI and NCCL primitives: 1) message size; 2) message count; 3) latency of a single communication operation; and 4) aggregate latency. These statistics are then compared to the end-to-end performance (images/second) reported by application codes like *tf_cnn_benchmarks*. An overview of *hvpof* and where it is positioned in the overall hierarchy is provided in Figure 1. We broadly divide operations into two types: 1) MPI/NCCL operations used by the DL framework, and 2) MPI operations used by the Horovod communication engine and response cache.

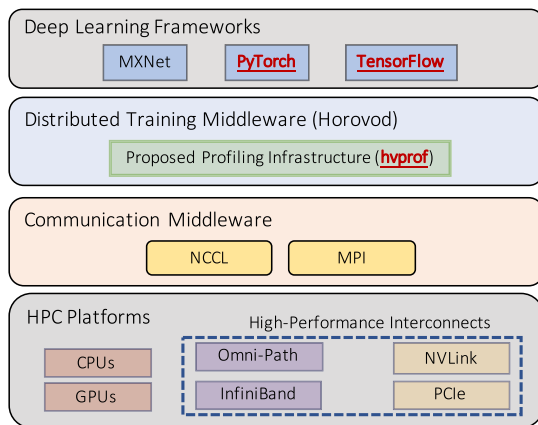


Figure 1. Overview of the proposed profiling infrastructure.

Proposed Padding Design

Based on characterization results presented earlier by Awan *et al.*,⁷ we observed that message sizes that are not a power-of-two (e.g., 321) cause inconsistent performance behavior on all interconnects. To address it, we propose a simple padding scheme that finds the next power-of-two size for a given tensor and appends it with additional dummy bytes. Thus, Horovod's fusion-buffer will now become an exact power-of-two sized buffer. We call MPI_Allreduce/ncclAllreduce on this new padded fusion-buffer instead of the original buffer. The Allreduce latency profiles presented in Section Impact of Padding Design show that this simple padding scheme can significantly reduce the performance instabilities. The main reason for poor non-power-of-two performance is because most MPI libraries implement collective communication algorithms optimized for power-of-two message sizes.

PERFORMANCE ANALYSIS

We provide a brief overview of the evaluation platforms and metrics followed by comprehensive sections on 1) end-to-end application performance; 2) low-level profiling analysis; and 3) impact of the proposed padding design.

Evaluation Platform

We utilized two well-known HPC clusters: 1) Skylake partition of TACC Stampede2 equipped with Intel *Omni-Path* interconnect and 2) Ohio Supercomputing Center (OSC) Pitzer equipped with Mellanox *InfiniBand*. For GPU-based training, we evaluated and profiled DL workloads on multiple Volta V100 GPUs connected using NVLink and IBM X-bus within a node an InfiniBand across nodes.

We used the following communication libraries: Intel MPI (2018), MVAPICH2 (2.3.1), MVA-PICH2-GDR (2.3.1), NCCL2 (2.4), Spectrum MPI (10.3), and Open MPI (4.0.1) with UCX (1.6.0).

Metrics

The main metric for end-to-end application performance is *images/second* (img/sec) for synchronous distributed training using Horovod for TensorFlow and PyTorch models. The low-level metric we have used is latency (time of each call) for the Allreduce operation. Message count and size have already been discussed in the earlier version of this article⁷ and have been omitted here.

End-to-End Performance

CPU Platforms An overview of end-to-end performance for ResNet-101 and Inception-v4 on

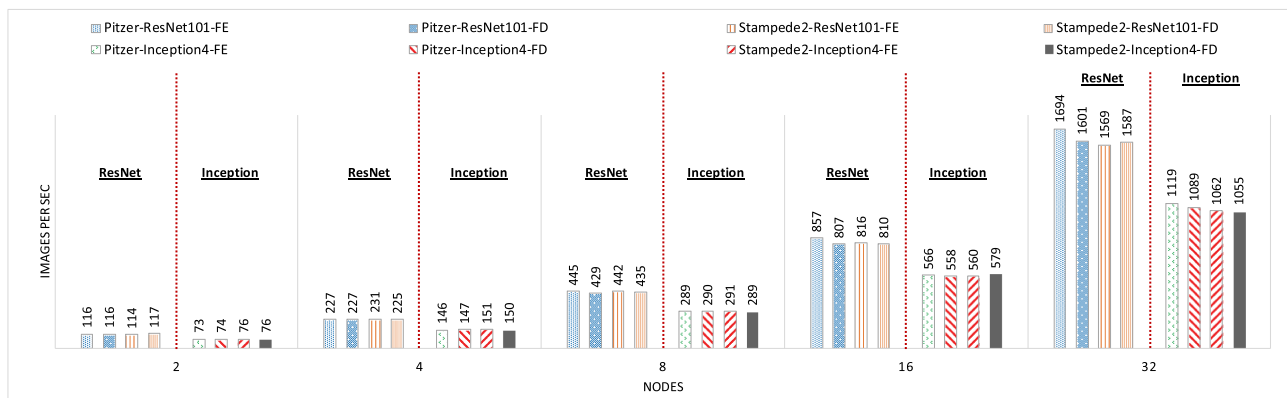


Figure 2. TensorFlow: Multinode training—FE and FD on Stampede2 and Pitzer.

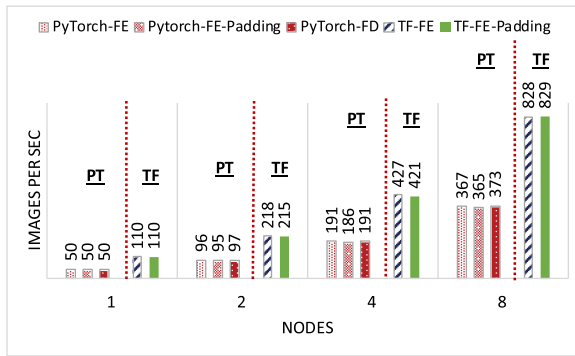
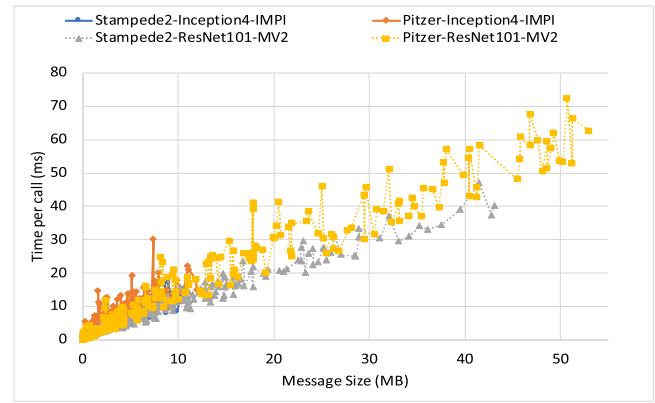


Figure 3. End-to-end performance comparison for TensorFlow (TF) and PyTorch (PT) on Stampede2: FD, FD, and fusion enabled with padding (FE-Padding).

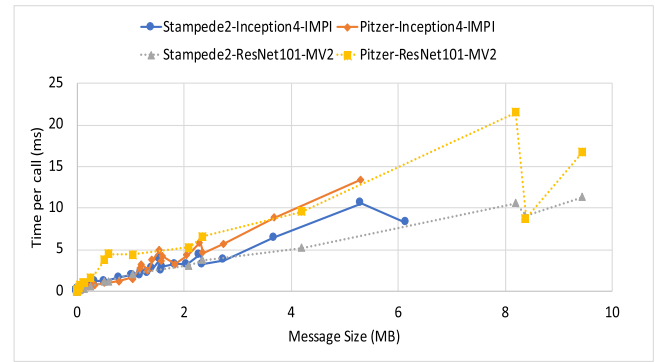
Pitzer (InfiniBand) and Stampede2 (Omni-Path) is shown in Figure 2. We use the Intel-optimized version of TensorFlow (v1.13) on all Intel CPU platforms to take advantage of MKL optimizations. For Pitzer, enabling tensor fusion (FE) results in better performance for both ResNet-101 and Inception-v4. On the other hand, for Stampede2, we observed that disabling fusion (FD) gives slightly better (up to 5%) performance for a higher number of nodes. The behavior of ResNet-101 and Inception-v4 is slightly different when tensor-fusion is enabled or disabled on Stampede2. PyTorch Versus TensorFlow With Padding: We now present the experiments performed to evaluate PyTorch as well as the new padding designs in *huprof*. Figure 3 shows how TensorFlow and PyTorch perform under different settings like fusion enabled (FE), fusion disabled (FD), etc. and how padding can impact the end-to-end performance for both cases. The data is presented only for the Stampede2 cluster.

Table 1. End-to-end performance on GPUs (only the best cases are shown for all configurations).

Comm. Library	NVLink (3 GPUs)	NVLink-Xbus-Infiniband (12 GPUs)
MV2-GDR	1129	4274
NCCL	1129	4388
Spectrum MPI	885	2772
OpenMPI	1013	1975



Fusion Enabled (FE)



Fusion Disabled (FD)

Figure 4. Allreduce latency for Inception-v4 training on Stampede2 (Omni-Path) and Pitzer (InfiniBand) using Intel MPI and MVAPICH2 with tensor-fusion enabled and disabled.

GPU Platforms GPU-based end-to-end performance behavior was evaluated with two different configurations.

- 1) GPUs connected with NVLink only — (G1)
- 2) GPUs connected with NVLink (same socket), X-Bus (across sockets), and InfiniBand (across nodes) — (G2).

For G1, we trained ResNet-50 using different communication libraries. The NVLink column of Table 1 shows the end-to-end numbers. The lower-level allreduce latency for all these is discussed further in Section Low-Level Profiling Analysis. We see that MVAPICH2 and NCCL2 are much better compared to others. Spectrum MPI is the slowest of all the communication libraries. We note that this high-level *img/sec* trend matches the low-level allreduce statistics collected using *huprof* as shown in Figure 5.

For G2, we trained ResNet-50 on two nodes in a 6-GPU/node configuration where three GPUs are on the same socket and are only connected

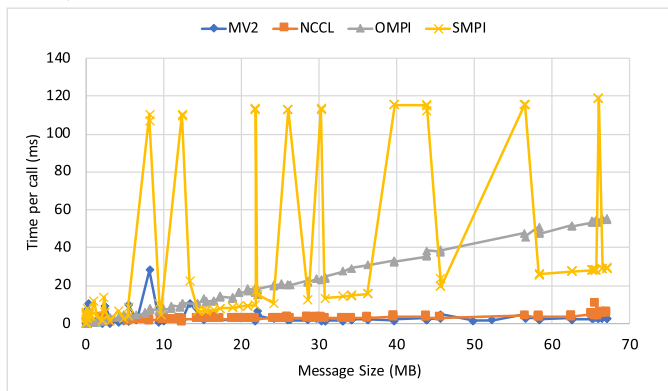


Figure 5. Allreduce latency for ResNet-50 training on three NVLink-connected V100 GPUs (without padding).

using NVLinks whereas the other three GPUs within the node are cross sockets and communication will take place using IBM's X-Bus interface between two POWER9 CPUs. Traffic across nodes will go over a socket-direct InfiniBand EDR HCA. We report the default numbers as well as numbers with the proposed padding design.

The trends for **G2** do not match **G1**. For example, under **G2**, we observed 4313 img/s with padding for MVAPICH2-GDR whereas only 4274 img/s without padding. On the other hand, NCCL2 offered 4362 img/s with padding, which is lower than 4388 img/s without padding. Spectrum MPI seems to perform better than Open MPI for this case and gives 2772 img/s without padding whereas only 2522 with padding. Open MPI performed the worst and gives 1975 img/s without padding and only 1835 img/s with padding designs.

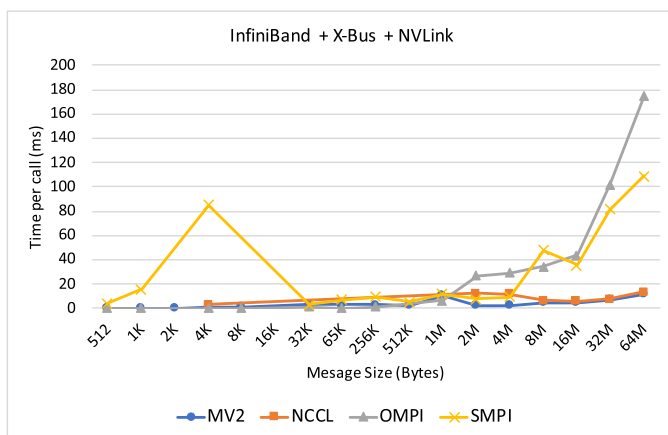


Figure 6. Allreduce latency for ResNet-50 training on two nodes in 6-GPU/node configuration (with padding).

Low-Level Profiling Analysis

We discuss several low-level profiles in this section. We profiled ResNet-101 and Inception-v4 training on Stampede2 and Pitzer using Intel MPI and MVAPICH2. We broadly divide the results into two combinations: 1) Tensor Fusion Enabled (**FE**); and 2) Tensor Fusion Disabled (**FD**). Both are shown in Figure 4.

Horovod's tensor fusion design causes the use of multiple nonpower-of-two message sizes, which results in performance variations (spikes) for both Intel MPI and MVAPICH2 on both clusters. On the other hand, disabling fusion restricts the message sizes to differ significantly less compared to fusion, which results in fewer spikes for both MPI libraries and on both clusters.

Figure 5 provides a three-GPU profiling analysis for ResNet-50 training on Volta V100 GPUs. We have two main cases: 1) GPUs connected with NVLink (on POWER9); and 2) GPUs connected with PCIe (on Intel). Broadly, we observed higher latency and more spikes for OpenMPI and SpectrumMPI compared to MVAPICH2-GDR and NCCL.

Impact of Padding Design

To further understand the behavior of GPU-based training, we performed the ResNet-50 training experiment on two-nodes with 6 GPUs/node and evaluate the impact of padding designs. As shown in Figure 6, we observe new trends for all four communication libraries. MVAPICH2-GDR and NCCL2 offer much smoother latency curves compared to Spectrum MPI and Open MPI. Latency for large-messages for both Spectrum MPI and Open MPI is significantly worse compared to MVAPICH2-GDR and NCCL2.

On the CPU side, we performed new experiments on eight nodes for both Pitzer and Stampede2 to study the impact of padding designs. Figure 7 shows the impact of padding on TensorFlow as well as PyTorch. We note that PyTorch is much slower compared to TensorFlow as reported earlier in end-to-end performance (Section End-to-End Performance). We can see from Figure 7 that the allreduce latency curves are not as inconsistent for PyTorch as it is for TensorFlow. This result can be confusing if the end-to-end number is not considered and one can wrongly conclude that TensorFlow's behavior is inconsistent.

RELATED WORK

Most related studies have focused on large-scale evaluation and characterization of DL workloads on GPUs.^{4,8} Some papers have focused on CPU based platforms as well.^{9,10} Large-scale training experiments for Intel CPUs are discussed by Codreanu *et al.*¹¹ The best practices guide¹² published by Intel provides data on multinode training with Intel-optimized TensorFlow. However, all these papers only provide end-to-end performance behavior for DL workloads. To the best of our knowledge, this article is the first study that provides not only end-to-end performance but also offers in-depth data about low-level metrics like number and time taken by allreduce calls used for synchronization and gradient communication. Furthermore, none of the published works explore the performance impact of interconnects like InfiniBand and Omni-Path on distributed training.

CONCLUSION

DL workloads are becoming increasingly important for HPC clusters equipped with cutting-edge CPUs and GPUs as well as interconnects like InfiniBand and Omni-Path. In this article, we highlighted challenges in evaluating and profiling DL workloads and how to overcome these through the proposed profiling infrastructure called *huprof*. We offer performance characterization for various combinations of DNNs like ResNet(s) and Inception variants on CPU and GPU systems with interconnects like InfiniBand, Omni-Path, X-Bus, and NVLink. We highlight that MPI as well as NCCL perform well for power-of-two message sizes but provide inconsistent performance for nonpower-of-two message sizes. We also attempt to mitigate this by designing and evaluating a padding design for Horovod. Even with the padding designs, we believe that there is an opportunity to redesign Horovod and/or communication runtimes to further optimize performance for these nonpower-of-two message sizes. In the future, we plan to investigate new designs that avoid the main problem with current padding design, i.e., adding more data due to padding and thereby increasing the traffic over interconnects.

ACKNOWLEDGMENTS

This work was supported in part by the National Science Foundation (NSF) under Grants

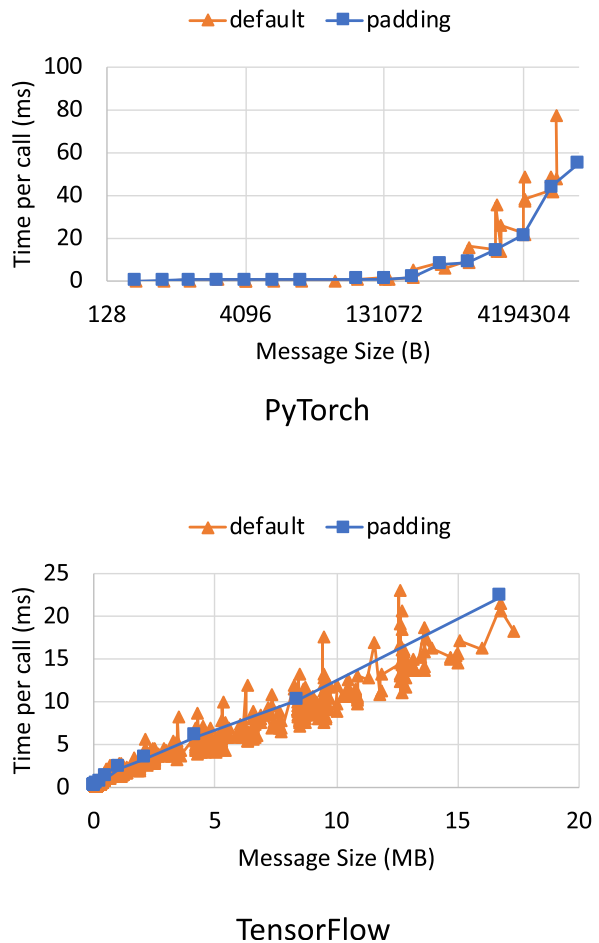


Figure 7. PyTorch allreduce profiler for eight nodes, 48 ppn, and ResNet-50.

#CNS-1513120, #ACI-1450440, #CCF-1565414, and #ACI-1664137.

REFERENCES

1. A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," 2018. [Online]. Available: <http://arxiv.org/abs/1802.05799>
2. S. A. Mojumder *et al.*, "Profiling DNN workloads on a volta-based DGX-1 system," in *Proc. IEEE Int. Symp. Workload Characterization*, Sep. 2018, pp. 122–133.
3. C. Pearson, I.-H. Chung, Z. Sura, W.-M. Hwu, and J. Xiong, "NUMA-aware data-transfer measurements for power/NVLink multi-GPU systems," in *Proc. Int. Conf. High Perform. Comput.*, 2018, pp. 448–454.
4. A. A. Awan, H. Subramoni, and D. K. Panda, "An in-depth performance characterization of CPU- and GPU-based DNN training on modern architectures," in *Proc. Mach. Learn. HPC Environ.*, 2017, pp. 8:1–8:8.

5. K. S. Khorassani, C.-H. Chu, H. Subramoni, and D. K. Panda, "Performance evaluation of MPI libraries on GPU-enabled OpenPOWER architectures: Early experiences," in *Proc. Int. Conf. High Perform. Comput.*, 2018.
6. Baidu Research, "Benchmarking deep learning operations on different hardware," 2016, Accessed: 2019/10/31 15:45:00. [Online]. Available: <https://github.com/baidu-research/DeepBench>
7. A. A. Awan, A. Jain, C.-H. Chu, H. Subramoni, and D. K. Panda, "Communication profiling and characterization of deep learning workloads on clusters with high-performance interconnects," in *Proc. 26th Symp. High-Perform. Interconnects*, 2019.
8. A. A. Awan, J. Bédorf, C.-H. Chu, H. Subramoni, and D. K. Panda, "Scalable distributed DNN training using TensorFlow and CUDA-aware MPI: Characterization, designs, and performance evaluation," in *Proc. 19th Annu. IEEE/ACM Int. Symp. Cluster, Cloud, Grid Comput.*, 2019, pp. 498–507.
9. S. Shi, Q. Wang, P. Xu, and X. Chu, "Benchmarking state-of-the-art deep learning software tools," in *Proc. 7th Int. Conf. Cloud Comput. Big Data*, Nov. 2016, pp. 99–104.
10. A. Viebke and S. Pillana, "The potential of the Intel (R) Xeon Phi for supervised deep learning," in *Proc. IEEE 17th Int. Conf. High Perform. Comput. Commun., 7th Int. Symp. Cyberspace Safety Secur. 12th Int. Conf. Embedded Softw. Syst.*, Aug. 2015, pp. 758–765.
11. V. Codreanu, D. Podareanu, and V. Saletore, "Large minibatch training on supercomputers with improved accuracy and reduced time to train," in *Proc. IEEE/ACM Mach. Learn. HPC Environ.*, Nov. 2018, pp. 67–76.
12. A. Bhandare *et al.*, "Best practices for scaling deep learning training and inference with Tensorflow on Intel Xeon processor-based HPC infrastructures," Connectivity Group & AI Products Group, Data Center Group Customer Solutions Technical Enabling, Intel Corporation, Tech. Rep., Jan. 2019.

Ammar Ahmad Awan is currently working toward the PhD degree at The Ohio State University, Columbus, OH, USA. He received the BS and MS degrees in computer science and engineering from the National University of Science and Technology, Islamabad, Pakistan, and Kyung Hee University, Seoul, South Korea, respectively. His current research focus lies at the intersection of high-performance computing libraries and deep learning frameworks. He previously worked on a java-based message passing interface (MPI) and nested parallelism with OpenMP and MPI for scientific applications. He has authored or coauthored 20 papers in conferences and journals related to these research areas. He actively contributes to various projects like MVAPICH2-GDR (high-performance MPI for GPU clusters, OMB (OSU Micro Benchmarks), and HiDL (high-performance deep learning). He is the lead author of the OSU-Caffe framework (part of HiDL project) that allows efficient distributed training of deep neural networks. Contact him at awan.10@osu.edu.

Arpan Jain is currently working toward the PhD degree at The Ohio State University, Columbus, OH, USA, working with Prof. D. K. Panda. He received the BS and MS degrees in information and communication technology from ABV—Indian Institute of Information Technology and Management Gwalior, Gwalior, India. He is currently a Graduate Research Assistant with the Network-Based Computing Laboratory, Columbus, OH, USA. His current research interests lie at the intersection of deep learning and high-performance computing. He is working on parallelization and distribution strategies for large-scale deep neural network (DNN) training. He previously worked on speech analysis, time series modeling, hyperparameter optimization using nature-inspired algorithms for DNN, and object recognition. He has authored or coauthored nine papers in conferences and journals related to these areas. He actively contributes to projects like HiDL (high-performance deep learning) and MVAPICH2-GDR (high-performance MPI for GPU clusters). Contact him at jain.575@osu.edu.

Ching-Hsiang Chu is currently working toward the PhD degree in computer science and engineering at The Ohio State University, Columbus, OH, USA. He received the BS and MS degrees in computer science and information engineering from the National Changhua University of Education, Changhua City, Taiwan, in 2010, and the National Central University, Taoyuan City, Taiwan, in 2012, respectively. His research interests include high-performance computing, GPU communication, and wireless networks. He is a student member of IEEE and ACM. Contact him at chu.368@osu.edu.

Hari Subramoni has been a Research Scientist with the Department of Computer Science and Engineering, The Ohio State University, Columbus, OH, USA, since September 2015. He received the PhD degree in computer science from The Ohio State University, Columbus, OH, USA, in 2013. His current research interests include high-performance interconnects and protocols, parallel computer architecture, network-based computing, exascale computing, network topology aware computing, QoS, power-aware LAN-WAN communication, fault tolerance, virtualization, big data, and cloud computing. He has authored or coauthored more than

50 papers in international journals and conferences related to these research areas. He is doing research and working on the design and development of MVAPICH2, MVAPICH2-GDR, and MVAPICH2-X software packages. He is a member of IEEE. Contact him at subramoni.1@osu.edu.

Dhableswar K. (DK) Panda is currently a Professor and University Distinguished Scholar of Computer Science and Engineering at The Ohio State University, Columbus, OH, USA. He has authored or coauthored more than 450 papers in major journals and international conferences. The MVAPICH2 (high-performance MPI over InfiniBand, iWARP, and RoCE) open-source software package, developed by his research group (<http://mvapich.cse.ohio-state.edu>), is currently being used by more than 3025 organizations worldwide (in 89 countries). This software has enabled several InfiniBand clusters to get into the latest TOP500 ranking during the last decade (including the current #3). More than 600 000 downloads of this software have taken place from the project's website alone. He is an IEEE Fellow and a member of ACM. Contact him at panda.2@osu.edu.



IEEE Security & Privacy magazine provides articles with both a practical and research bent by the top thinkers in the field.

- stay current on the latest security tools and theories and gain invaluable practical and research knowledge,
- learn more about the latest techniques and cutting-edge technology, and
- discover case studies, tutorials, columns, and in-depth interviews and podcasts for the information security industry.



computer.org/security